



Aula 3

Redes Convolucionais e Transfer Learning

Tópicos Avançados em Inteligência Artificial · UFABC

Parte 1 — Visão Computacional

Como representar imagens e que tarefas resolver com DL.

Tensores e imagens

Tensor = array N-dimensional de números:

rank 0	rank 1	rank 2	rank 3	rank 4
escalar	vetor	matriz	cubo	rank-4
				
42 um número	[a, b, c, d] lista de números	imagem cinza $H \times W$	imagem RGB $H \times W \times 3$	batch RGB $N \times H \times W \times 3$

Imagem em escala de cinza

Matriz $H \times W$ — cada pixel: 0–255

Imagem colorida (RGB)

Tensor $H \times W \times 3$ — 3 canais (R, G, B)

Batch de imagens

Tensor $N \times H \times W \times 3$

Tarefas de visão computacional

Classificação



qual é a classe?

Classificação

Uma classe por imagem

Localização

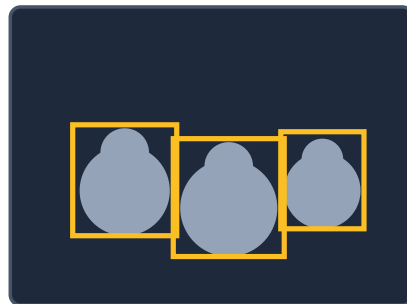


classe + caixa

Deteção

Várias caixas + classes

Deteção de objetos



várias classes/caixas

Segmentação

Classe por pixel

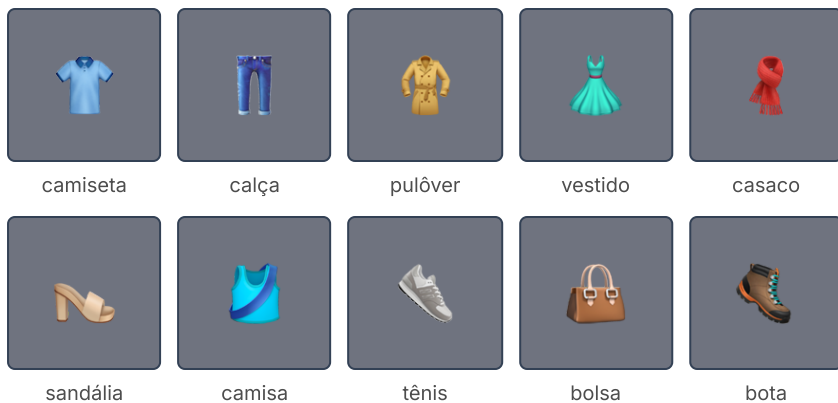
Segmentação semântica



classe por pixel

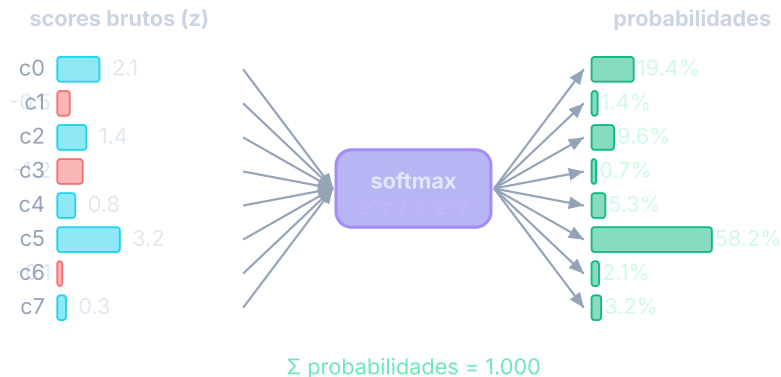
Classificação multiclasse — softmax

Fashion-MNIST: 70k imagens 28×28, 10 categorias.



Para N classes, a camada de saída usa **softmax**:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$



Saídas $\in (0, 1)$ com soma exatamente 1.

Casando saída com loss

Variável de saída	Camada de saída	Keras / TF	PyTorch
Número real	linear (1 neurônio)	<code>mse</code>	<code>nn.MSELoss</code>
Probabilidade binária	sigmoide (1 neurônio)	<code>binary_crossentropy</code>	<code>nn.BCELoss</code> *
Vetor de probabilidades	softmax (N neur.)	<code>categorical_crossentropy</code>	<code>nn.CrossEntropyLoss</code> †
Idem, rótulos inteiros	softmax (N neur.)	<code>sparse_categorical_crossentropy</code>	<code>nn.CrossEntropyLoss</code>

* preferir `nn.BCEWithLogitsLoss` (mais estável numericamente) † inclui log-softmax: não adicione `nn.Softmax` à saída

⚠ Loss errada para o tipo de rótulo é uma das fontes mais comuns de bug.

Baseline: rede densa no Fashion-MNIST

Keras

```
inp = keras.Input(shape=(28, 28))
x = keras.layers.Flatten()(inp)
x = keras.layers.Dense(128, 'relu')(x)
x = keras.layers.Dropout(0.3)(x)
x = keras.layers.Dense(64, 'relu')(x)
out = keras.layers.Dense(10, 'softmax')(x)
model = keras.Model(inp, out)
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 64), nn.ReLU(),
    nn.Linear(64, 10),
)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())
# CrossEntropyLoss já inclui softmax!
```

Baseline de ~88 % com camadas densas. Podemos fazer melhor?

Parte 2 — Por que camadas densas não bastam

O problema de achatar imagens.

O problema com Flatten

- Imagem colorida de celular: **3024 × 3024 × 3 pixels**
- Achatar e conectar a uma camada de 100 neurônios → **≈ 2,7 bilhões de parâmetros**
- Computacionalmente inviável, precisa de enormes volumes de dados, propicia overfitting

Ao achatar, perdemos a **estrutura espacial** — pixels vizinhos carregam informação local que a camada densa ignora.

Se um padrão aparece em posições diferentes da imagem, a rede densa precisa **aprendê-lo de novo** para cada posição. Filtros convolucionais **aprendem uma vez e reutilizam** em qualquer posição.

Flatten: o que se perde

Imagem 4×4 (original)

A	B	.	.
C	.	.	.
.	.	.	.
.	.	.	.

A-B: vizinhos → ✓

A-C: vizinhos ↓

→
Flatten

Vetor 1×16 (após Flatten)

A	B	.	.	C	.	.	.	.	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

A-B: ainda vizinhos ✓

A-C: agora 4 posições apart ✗

Explosão de parâmetros

$224 \times 224 \times 3 \rightarrow 128$ neurônios = **19M params**
só nessa camada

Perde invariância

Gato no canto $\neq$ gato no centro — a rede trata como entradas distintas

Perde adjacência 2D

Vizinhos verticais ficam *largura* posições de distância no vetor

Parte 3 — Filtros Convolucionais

Como detectar padrões visuais de forma eficiente.

O filtro convolucional

- Um **filtro** é uma pequena matriz de números (ex.: 3×3)
- Deslizando sobre a imagem, detecta um tipo de padrão visual
- Uma **camada convolucional** tem múltiplos filtros; cada um aprende um padrão diferente

Borda vertical:	Borda horizontal:
1 0 -1	1 1 1
1 0 -1	0 0 0
1 0 -1	-1 -1 -1

Analogia com neurônio denso:

- Neurônio denso → conectado a **todos** os pixels
- Filtro convolucional → conectado a uma **janela local** e desliza com os **mesmos pesos**

Benefícios:

- **Muito menos parâmetros**
- Preserva **adjacência espacial**
- **Invariância à translação** — detecta o mesmo padrão em qualquer posição

A operação de convolução

Passos:

1. Posiciona o filtro sobre uma janela da imagem
2. Multiplica elemento a elemento e soma → um escalar
3. Desliza (stride) e repete
4. Resultado: um **feature map**
5. Aplica ReLU para não-linearidade

Janela 3×3:	Filtro (borda vertical):	
1 2 3	1 0 -1	
4 5 6 ×	1 0 -1	= (1+4+7)-(3+6+9) = -6
7 8 9	1 0 -1	

Valor alto no feature map = filtro **detectou** o padrão naquela região.

Ao empilhar camadas:

- Cam. 1 → bordas e texturas simples
- Cam. 2 → cantos, curvas
- Cam. 3+ → partes de objetos, objetos completos

Convolução passo a passo

Entrada 6×6 (trecho) — filtro 3×3 desliza com stride 1:

```

  . . .
  | 1 | 2 | 3 | 0 1
  | 4 | 5 | 6 | 1 0 → posição (0,0) → -6
  | 7 | 8 | 9 | 0 1
  . . .

```

```

. . .
. | 2 | 3 | 0 | → posição (0,1) → 3
. | 5 | 6 | 1 |
. | 8 | 9 | 0 |
. . .

```

Filtro (borda vertical):

```

1  0 -1
1  0 -1
1  0 -1

```

Feature map resultante (4×4):

```

-6  3  ...
...  ...

```

Tamanho do output: $(H - F + 1) \times (W - F + 1)$

Entrada 6×6, filtro 3×3 → output **4×4**

Com **padding=1** → output **6×6** (mesma resolução)

O que os filtros detectam

Borda vertical

1	0	-1
1	0	-1
1	0	-1

Alta resposta onde intensidade muda da esquerda para direita

Borda horizontal

1	1	1
0	0	0
-1	-1	-1

Alta resposta onde intensidade muda de cima para baixo

Blur (suavização)

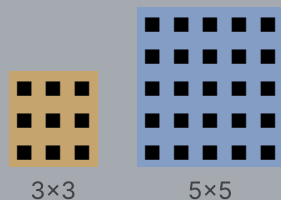
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Média dos vizinhos → suaviza ruído e detalhes

Em CNNs, **os valores do filtro não são definidos à mão** — são **aprendidos pelo backprop**. A rede descobre sozinha quais padrões são úteis para a tarefa. Camadas profundas combinam bordas → cantos → partes → objetos.

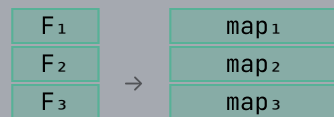
Parâmetros de uma camada convolucional

Tamanho do filtro



3x3 preferido — bom custo-benefício

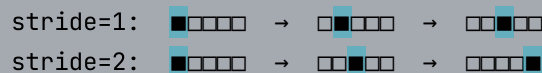
Número de filtros (K)



K filtros K feature maps

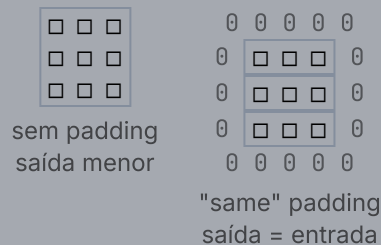
K=32, 64, 128... (hiperparâmetro)

Stride



stride>1 reduz dimensão sem pooling

Padding



32 filtros 3x3 x 3 canais: $32 \times (3 \times 3 \times 3 + 1) = 896$ params

Densa equivalente ($224 \times 224 \times 3 \rightarrow 32$): $\approx$ **4,8 M params**

Parte 4 — Pooling

Reduzindo dimensões sem perder o essencial.

Max Pooling

- Divide o feature map em janelas (ex.: 2×2)
- Retém apenas o **valor máximo** de cada janela
- Reduz a resolução espacial pela metade
- Menos parâmetros nas camadas seguintes

Intuição: se uma feature existe **em qualquer lugar** da janela, o max pooling a preserva. Confere robustez a pequenos deslocamentos.

Feature map (4×4):

1	3		2	4
5	6		1	2
-----+				
7	2		3	1
4	1		5	2

MaxPool 2×2 :

6	4
7	5

Max Pooling vs Average Pooling

Max Pooling — preserva a *presença* da feature

Feature map (4×4):

1	3	2	4
5	6	1	2
7	2	3	1
4	1	5	2

→ MaxPool 2×2:

6	4
7	5

Máximo de cada janela: "este padrão **existe** aqui?"

MaxPool — padrão em CNNs para visão; detecta a presença de features

Average Pooling — preserva a *intensidade média*

Feature map (4×4):

1	3	2	4
5	6	1	2
7	2	3	1
4	1	5	2

→ AvgPool 2×2:

3.75	2.25
3.5	2.75

Média de cada janela: "qual a intensidade **típica** desta região?"

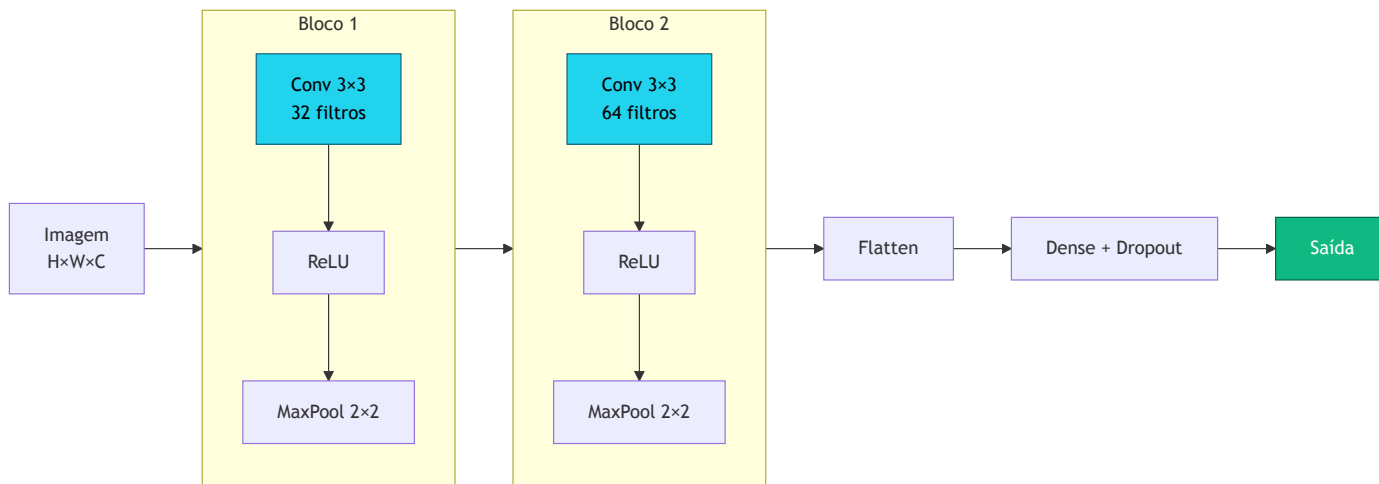
GlobalAvgPool — usado antes da cabeça de classificação em redes modernas (ex.: ResNet); produz um vetor por canal sem Flatten

Parte 5 — Arquitetura CNN

Combinando convoluções, pooling e camadas densas.

Blocos convolucionais

Uma CNN é construída por **blocos convolucionais** empilhados — cada bloco tem **mais profundidade** e **menor resolução** que o anterior — seguidos de camadas densas:



Empilhando camadas — hierarquia de padrões

Por que empilhar blocos convolucionais?

- Cada camada vê uma **janela local** da camada anterior
- Camadas profundas têm **campo receptivo maior** — enxergam mais da imagem
- A rede aprende uma **hierarquia de representações**, do simples ao complexo

Visualização empírica (Zeiler & Fergus, 2014): filtros de redes treinadas mostram progressão clara — camadas iniciais detectam bordas e gradientes, camadas intermediárias detectam texturas e partes, camadas profundas detectam objetos completos.

Camada 1 → bordas e gradientes
/ \ - | /
Camada 2 → texturas e cantos
◻ ▣ ◆ ▨
Camada 3 → partes de objetos
👁 🍷 🦴
Camada 4 → objetos / conceitos
🐱 🚗 🌸

Profundidade → abstração crescente.

Transfer learning funciona porque estas representações **generalizam** para novos domínios.

CNN para Fashion-MNIST

Keras

```
inp = keras.Input(shape=(28, 28, 1))
x = keras.layers.Conv2D(
    32, 3, activation='relu', padding='same')(inp)
x = keras.layers.MaxPooling2D()(x)
x = keras.layers.Conv2D(
    64, 3, activation='relu', padding='same')(x)
x = keras.layers.MaxPooling2D()(x)
x = keras.layers.Flatten()(x)
x = keras.layers.Dense(128, activation='relu')(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(10, activation='softmax')(x)
model = keras.Model(inp, out)
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
model = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 128), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 10),
)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())
```

CNN atinge ~92 % no Fashion-MNIST, contra ~88 % da rede densa.

Conexões Residuais (*Skip Connections*)

O problema de redes muito profundas

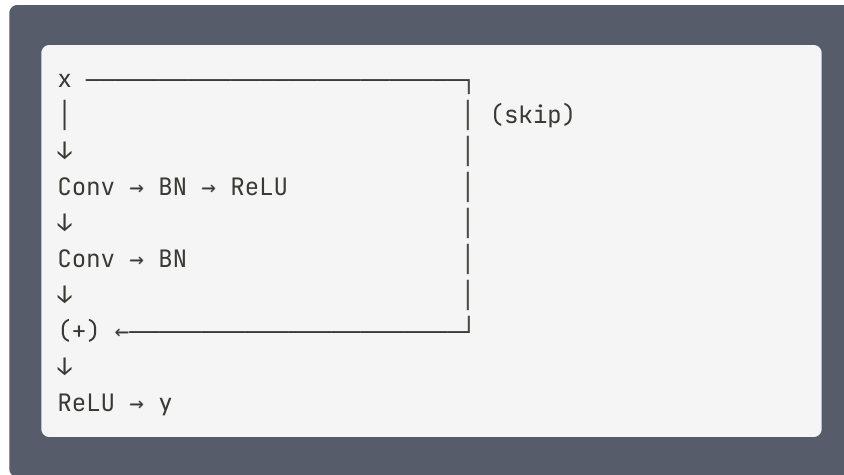
- Gradiente se dissipa camada a camada (*vanishing gradient*)
- Redes com >20 camadas convergiam pior do que redes rasas

Ideia (He et al., ResNet 2015): permitir que o sinal flua diretamente pulando camadas.

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, W) + \mathbf{x}$$

Em vez de aprender $\mathbf{y}$, o bloco aprende o **resíduo** $\mathcal{F} = \mathbf{y} - \mathbf{x}$.

Se $\mathcal{F} \approx 0$, o bloco age como **identidade** — nunca piora o resultado anterior.



BN = *Batch Normalization*: normaliza as ativações de cada camada (média 0, variância 1 por batch), acelerando a convergência e estabilizando o treino.

$$y = \mathcal{F}(\mathbf{x}, W) + \mathbf{x}$$

Gradiente flui pelo skip sem depender de $\mathcal{F} \rightarrow$ ResNet-152 treina com sucesso.

Parte 6 — Transfer Learning

Reutilizando o que foi aprendido em milhões de imagens.

Duas tendências que viabilizaram o transfer learning

Tendência 1 — Arquiteturas especializadas:

Tipo de dado	Arquitetura
Qualquer	Conexões residuais (ResNet)
Imagens	Camadas convolucionais
Sequências (texto, áudio, genes)	Transformers

Tendência 2 — Modelos pré-treinados disponíveis:

Usando essas arquiteturas, pesquisadores treinaram redes de alto desempenho em grandes datasets reais. Inúmeros modelos estão publicamente disponíveis.

A ideia central do transfer learning

- **ImageNet**: milhões de imagens de 1000 categorias do cotidiano
- Uma rede treinada no ImageNet desenvolve representação **hierárquica** de objetos visuais:
 - Camadas iniciais → bordas e texturas
 - Camadas intermediárias → partes de objetos
 - Camadas finais → objetos completos

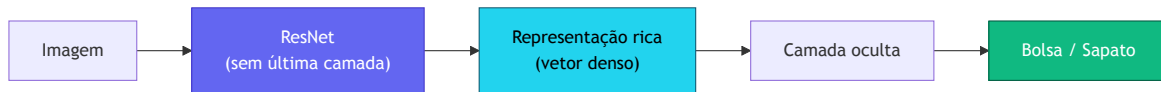
Problema: temos apenas ~100 imagens de bolsas e sapatos. Uma CNN do zero não vai generalizar.

Solução: pegar uma ResNet treinada no ImageNet e **reutilizar** o que ela já aprendeu sobre imagens do cotidiano.

Transfer learning = **personalizar** uma rede pré-treinada para um novo problema.

ResNet "decapitada" como extrator de features

1. ResNet original classifica 1000 categorias — a última camada não nos serve
2. Removemos a última camada ("**headless**" ResNet)
3. Passamos nossas imagens → saída é uma representação densa e rica
4. Treinamos uma rede pequena em cima dessa representação



Com apenas ~100 imagens, essa abordagem já produz alta acurácia.

Feature extraction vs. Fine-tuning

Feature extraction

- Pesos da base pré-treinada ficam **congelados**
- Treina apenas a cabeça nova
- Mais rápido, funciona com poucos dados
- Indicado quando o dataset-alvo é pequeno e similar ao ImageNet

Fine-tuning

- Conecta base + cabeça e treina **tudo end-to-end**
- Deve partir dos pesos pré-treinados (não do zero)
- Usa learning rate menor para não "esquecer"
- Melhor quando há dados suficientes

Catastrophic Forgetting

O problema

- Ao fazer fine-tuning, os gradientes da nova tarefa **sobrescrevem** os pesos importantes para a tarefa original
- A rede "esquece" o que aprendeu no pré-treino — perde a representação de features genéricas
- Quanto maior o learning rate e mais camadas descongeladas, mais severo o esquecimento

Sintoma: acurácia no dataset-alvo sobe rapidamente, mas o modelo perde capacidade de generalização — especialmente com poucos dados.

Estratégias para mitigar

Learning rate baixo

Fine-tuning com LR 10–100× menor que o pré-treino original (ex.: $1e-4$ em vez de $1e-2$)

Descongelamento gradual

Descongela as camadas de cima para baixo, uma a uma, ao longo do treino

Learning rates discriminativos

Camadas mais próximas da entrada recebem LR menor; camadas da cabeça recebem LR maior

Feature extraction primeiro

Treina só a cabeça até convergir; depois descongela para fine-tuning

Transfer learning em código

Keras

```
base = keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3),
)
base.trainable = False # congela

inp = keras.Input(shape=(224, 224, 3))
x = base(inp, training=False)
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dense(64, activation='relu')(x)
out = keras.layers.Dense(2, activation='softmax')(x)

model = keras.Model(inp, out)
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
import torchvision.models as models

base = models.resnet50(weights='IMAGENET1K_V2')

# Congela todos os pesos
for p in base.parameters():
    p.requires_grad = False

# Substitui a cabeça de classificação
n_feat = base.fc.in_features
base.fc = nn.Sequential(
    nn.Linear(n_feat, 64),
    nn.ReLU(),
    nn.Linear(64, 2),
)

optimizer = optim.Adam(base.fc.parameters())
criterion = nn.CrossEntropyLoss()
```

Onde encontrar modelos pré-treinados



TensorFlow Hub

tensorflow.org/hub

Modelos Keras prontos para uso



PyTorch Hub

pytorch.org/hub

ResNet, EfficientNet, BERT...



Hugging Face Hub

huggingface.co/models

+500k modelos: visão, NLP, áudio

Parte 7 — Aprendizado Auto-Supervisionado

Por que pré-treinar sem rótulos?

O problema

- Rotular dados é **caro e lento**
- Datasets anotados: ImageNet (1.2 M), CIFAR-10 (60 K)
- Dados não rotulados: **internet inteira**

A ideia

- Treinar o backbone em uma **tarefa auxiliar** criada a partir dos próprios dados
- Sem humano; o rótulo é gerado automaticamente
- Backbone aprendido é reutilizado (feature extraction / fine-tuning) com poucos exemplos rotulados

Aprendizado Auto-Supervisionado (SSL) = supervisão vem do próprio dado, não de anotadores humanos.

A tarefa auxiliar é chamada de **tarefa pretexto** (*pretext task*).

Tarefas Pretexto

Uma tarefa pretexto cria pares **(entrada, rótulo)** automaticamente, forçando o backbone a aprender representações úteis.

Previsão de Rotação

Rotaciona a imagem em 0°/90°/180°/270°.
Prevê qual rotação foi aplicada.

Gidaris et al., 2018 — RotNet

Patches Embaralhados

Divide em 9 patches e embaralha. Prevê a permutação original.

Noroozi & Favaro, 2016 — Jigsaw

Reconstrução Mascarada

Mascara ~75% dos patches. Reconstrói os pixels mascarados.

He et al., 2022 — MAE

Aprendizado Contrastivo

Maximiza similaridade entre visões da mesma imagem; minimiza entre diferentes.

Chen et al., 2020 — SimCLR

Colorização

Recebe imagem em escala de cinza. Prevê as cores (canais a e b do espaço Lab).

Zhang et al., 2016

Predição de Contexto

Extrai dois patches aleatórios. Prevê a posição relativa entre eles (8 direções).

Doersch et al., 2015

Denoising Autoencoder

Corrompe a entrada com ruído. Reconstrói a imagem original limpa.

Vincent et al., 2008 — DAE

Previsão de Rotação (RotNet)

Como funciona

1. Pega uma imagem qualquer (sem rótulo)
2. Aplica uma das 4 rotações: 0° , 90° , 180° , 270°
3. Backbone extrai features; cabeça de classificação prevê qual rotação (4 classes)
4. Loss: Cross-Entropy padrão



Por que funciona? Para acertar a rotação, o modelo precisa entender *o que está na imagem* — a orientação natural de objetos. Isso força o backbone a aprender features semânticas.

Patches Embaralhados (Jigsaw)

Procedimento

1. Divide a imagem em uma grade 3×3 (9 patches)
2. Embaralha os patches com uma permutação aleatória π
3. Backbone processa cada patch; cabeça prevê π de um conjunto pré-definido
4. Backbone aprende relações espaciais e contexto semântico

Original:

1	2	3
4	5	6
7	8	9

→

Embaralhado:

7	4	2
1	9	5
3	6	8

modelo deve prever: $\pi = [7, 4, 2, 1, 9, 5, 3, 6, 8]$

Intuição: Para remontar a imagem, o modelo deve entender **o que pertence junto** — conceito-chave para reconhecimento de objetos.

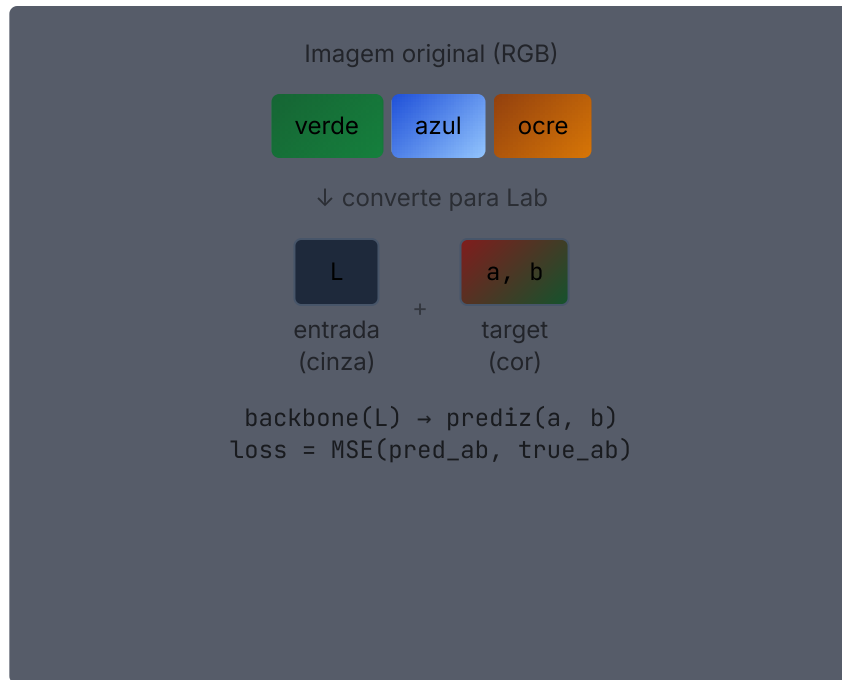
Colorização

Procedimento

1. Converte a imagem para o espaço de cores **Lab**
 - Canal **L** = luminosidade (escala de cinza)
 - Canais **a**, **b** = cromaticidade (cor)
2. Fornece apenas o canal **L** ao backbone
3. Backbone prevê os canais **a** e **b**
4. Loss: erro quadrático médio nos canais de cor

Por que funciona? Para colorir corretamente, o modelo precisa identificar o *que* é cada objeto — grama é verde, céu é azul, pele tem tons específicos. Isso força aprendizado semântico.

Vantagem: rótulo vem da imagem original — nenhuma anotação humana necessária. Qualquer foto colorida serve como dado de treino.



Limitação: ambiguidade de cores (carros podem ser de qualquer cor). Modelos aprendem a prever cores "seguras" médias; versões recentes usam distribuições probabilísticas.

Predição de Contexto (*Relative Patch Position*)

Procedimento (Doersch et al., 2015)

1. Extraí o patch **central** da imagem
2. Extraí um patch **vizinho** em uma das 8 posições ao redor
3. Backbone processa ambos os patches
4. Cabeça prevê qual das **8 direções** o vizinho ocupa
5. Loss: Cross-Entropy (8 classes)

Por que funciona? Para acertar a posição relativa, o modelo precisa entender partes de objetos e suas relações espaciais — ex.: olhos ficam acima do nariz, rodas ficam abaixo do carro.

Posições possíveis (8 direções)



```
backbone(patch_ref, patch_viz) → softmax(8)  
prevê: posição 2 (acima) ✓
```

Masked Autoencoders (MAE)

Ideia (He et al., 2022)

- Divide a imagem em patches de 16×16
- Mascara ~75% dos patches aleatoriamente
- **Encoder** (ViT) processa apenas patches visíveis
- **Decoder** leva reconstrói os pixels mascarados
- Loss: MSE nos pixels mascarados

imagem de entrada (patches):

■ ■ ■ ■ ■ ■ ■ ■
■ ■ ■ ■ ■ ■ ■ ■ (64 patches)

mascaramento (75%):

⊗ ■ ⊗ ⊗ ■ ⊗ ■ ⊗
⊗ ⊗ ■ ⊗ ⊗ ■ ⊗ ⊗ ⊗ = mascarado

Encoder → processa apenas ■

Decoder → reconstrói ⊗

→ backbone aprende representação
densa das partes visíveis

Por que 75%? Máscaras muito pequenas permitem interpolação trivial.
75% força compreensão global da cena.

Resultado: MAE pré-treina ViT-L em ImageNet em ~31 h (8 GPUs A100)
e atinge SOTA em classificação com fine-tuning.

Aprendizado Contrastivo (SimCLR)

Princípio

- Cria 2 **visões aumentadas** da mesma imagem (crop, flip, jitter de cor...)
- Maximiza similaridade entre visões da **mesma** imagem
- Minimiza similaridade com **outras imagens** no batch

NT-Xent Loss (InfoNCE):

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

τ = temperatura; z = embedding projetado

Batch grande → mais negativos → melhor sinal. SimCLR usa batch 4096.

```
imagem x
├─ aug1 → encoder → g(·) → z1
└─ aug2 → encoder → g(·) → z2 ┘ → max sim(z1, z2)
```

outras imagens no batch:

```
x' → encoder → g(·) → z'
                                → min sim(z1, z')
```

encoder: pesos compartilhados

g(·): projector, descartado
após pré-treino

Denoising Autoencoder (DAE)

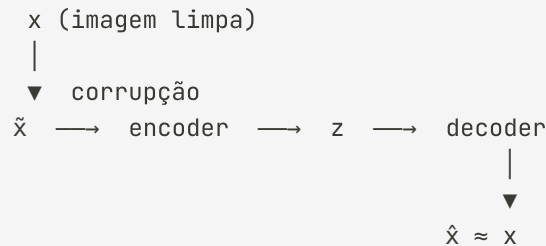
Ideia central (Vincent et al., 2008)

- Corrompe a entrada: ruído gaussiano, *salt-and-pepper*, apagamento de regiões
- O modelo aprende a mapear a versão corrompida $\tilde{x}$ de volta à original x
- O encoder é forçado a capturar estrutura semântica — não pode memorizar ruído

Loss de reconstrução:

$$\mathcal{L} = \|x - \text{dec}(\text{enc}(\tilde{x}))\|^2$$

Diferença do MAE: DAE usa ruído contínuo em pixels; MAE mascara patches inteiros. MAE escala melhor para ViTs.



tipos de ruído:

- gaussiano: $\tilde{x} = x + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$
- mascaramento: pixels $\rightarrow 0$ com prob. p
- salt-pepper: pixels $\rightarrow 0$ ou 255

Comparação dos Métodos SSL

Método	Tipo de sinal	Arquitetura	Pontos fortes
RotNet	Classificação (4 classes)	CNN	Simples; boa para pretraining rápido
Jigsaw	Classificação de permutação	CNN	Aprende relações espaciais
Colorização	Regressão (cores Lab)	CNN encoder-decoder	Aprende semântica de objetos naturais
Pred. Contexto	Classificação (8 direções)	CNN	Aprende layout espacial da cena
DAE	Reconstrução (ruído → limpo)	CNN encoder-decoder	Robusto a corrupções; pioneiro histórico
MAE	Reconstrução de pixels	ViT + decoder	SOTA em ViTs; eficiente
SimCLR	Contrastivo (similaridade)	CNN/ViT	Independente de rótulos; muito flexível
DINO	Auto-distilação	ViT	Features emergentes de segmentação

Métodos geradores (MAE, Jigsaw): o modelo reconstrói/ordena dados; aprendem geometria e textura detalhada.

Métodos contrastivos (SimCLR, DINO): o modelo aprende invariâncias; aprendem semântica de alto nível.

Recapitulando

Tensor $H \times W \times 3$ — representação de imagens coloridas;
batch: $N \times H \times W \times 3$

Flatten + Dense — baseline simples, mas explode em parâmetros e perde estrutura espacial

Max Pooling — downsampling que preserva presença de features; robustez a deslocamentos

Feature extraction — base congelada + nova cabeça;
funciona com poucos dados

Tarefa pretexto — rótulo gerado automaticamente do dado;
força backbone a aprender representações úteis

Softmax + Cross-Entropy — saída multiclasse; loss correta para N categorias

Filtro convolucional — janela local com pesos compartilhados; aprende uma vez, aplica em toda a imagem

CNN — blocos Conv+ReLU+Pool; profundidade cresce, resolução decresce

Fine-tuning — treina tudo end-to-end partindo dos pesos pré-treinados

SSL — RotNet, Jigsaw, MAE, SimCLR; pré-treino sem anotações humanas

Próxima aula

- **Sequências e linguagem** — como representar texto para redes neurais
- **Embeddings** — vetores densos para palavras e tokens
- **RNNs e LSTMs** — redes que processam sequências passo a passo
- **Attention e Transformers** — o mecanismo que domina NLP moderno
- **BERT e GPT** — pré-treinamento e fine-tuning para tarefas de linguagem

Obrigado! Perguntas?

Adaptado livremente de *15.773 Hands-on Deep Learning — Lecture 04* (MIT OpenCourseWare, 2024) — material original em inglês de Vivek Farias e Rama Ramakrishnan, distribuído sob os termos do MIT OCW.

Para mais informações: <https://ocw.mit.edu/terms>